

Redes Neurais

Alberto Paccanaro

EMap – FGV

<https://paccanarolab.org>

O material e as imagens nestes slides são de (ou adaptados de):

C. Bishop, Reconhecimento de Padrões e Aprendizado de Máquina, Springer, 2006

Vimos combinações lineares de funções de base **FIXAS**

regressão linear

- combinações lineares de funções de base **não lineares** fixas, $\phi_j(\mathbf{x})$:

$$\phi_j : \mathbb{R}^D \rightarrow \mathbb{R}$$

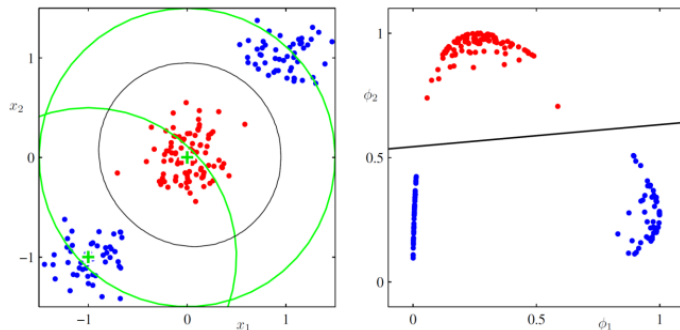
$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x})$$

As funções de base não lineares permitem que $y(\mathbf{x}, \mathbf{w})$ seja não linear !

classificação linear

Podemos aplicar transformações das entradas $\phi(\mathbf{x})$

O problema de classificação pode se tornar mais fácil



Regressão Logística (2 classes)

$$y(\phi) = \sigma(\mathbf{w}^T \phi)$$

Um sigmóide logístico atuando em uma função linear do **vetor de características (features)** ϕ

Que tal adaptar as funções de base aos dados?

- **FIXE A FORMA**, mas não o número:

Máquinas de vetores de suporte: defina funções de base centradas nos datapoints de treinamento e, em seguida, selecione um subconjunto deles durante o treinamento

- **FIXE O NÚMERO**, mas não a forma:

Redes neurais: o número de funções de base é fixo, mas elas se adaptam

- use formas paramétricas para as funções de base e os valores dos parâmetros são adaptados durante o treinamento
- Multilayer perceptron: múltiplas camadas de modelos de regressão logística 😊

Roteiro

- 
1. A ideia, as redes
 2. Funções de erro e funções de ativação
 3. Treinamento de rede neural
 4. O algoritmo de retropropagação (backpropagation)
 5. Regularização

Redes Neurais, em geral (feed-forward NN)

Modelos lineares para regressão e classificação:

$$y(\mathbf{x}, \mathbf{w}) = f \left(\sum_{j=1}^M w_j \phi_j(\mathbf{x}) \right)$$

Regressão: f é identidade

Classificação: f é não linear (ex. σ)

Ideia : fazer com que as funções de base $\phi_j(\mathbf{x})$ dependam de parâmetros e permitir que esses parâmetros sejam ajustados durante o treinamento (junto com os coeficientes w_j).

As redes neurais usam funções de base com forma similar

cada função de base é ela própria uma função não linear de uma combinação linear das entradas, onde os coeficientes na combinação linear são parâmetros adaptativos.

Uma série de transformações das entradas

1. Construa M combinações lineares das variáveis de entrada $x_1, \dots, x_D$:

$$a_j = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}$$

2. Cada combinação é transformada usando uma função de ativação não linear diferenciável $h(\cdot)$:

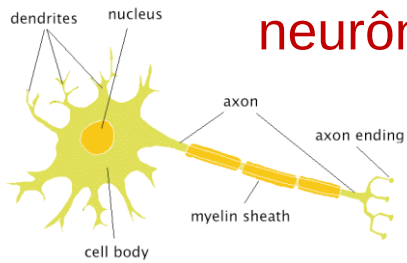
$$z_j = h(a_j)$$

3. Esses valores são novamente combinados linearmente:

$$a_k = \sum_{j=1}^M w_{kj}^{(2)} z_j + w_{k0}^{(2)}$$

Esta função complexa
pode ser representada
na forma de um
diagrama de rede...

4. ... e assim por diante, até a última função de ativação.



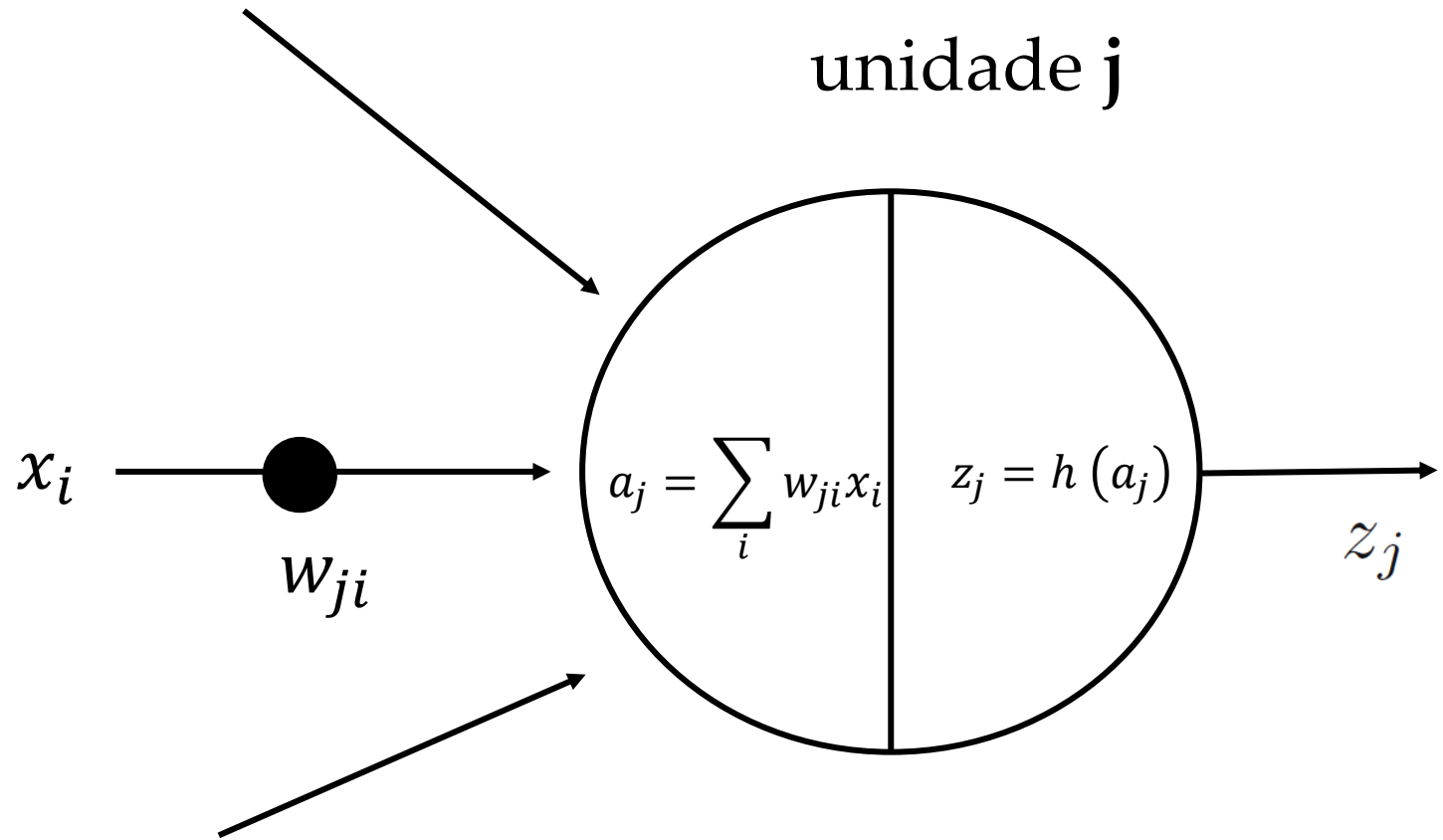
neurônio

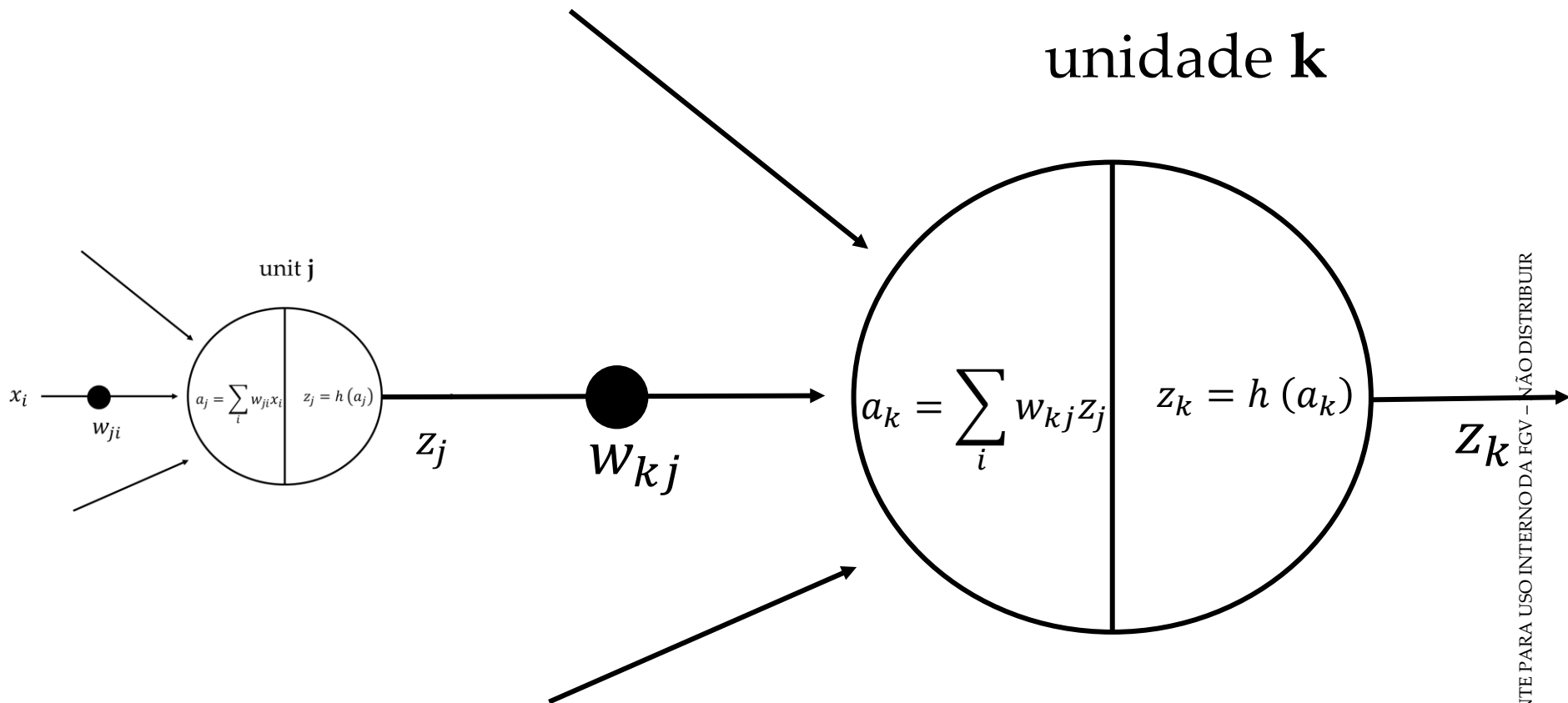
$$a_j = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}$$

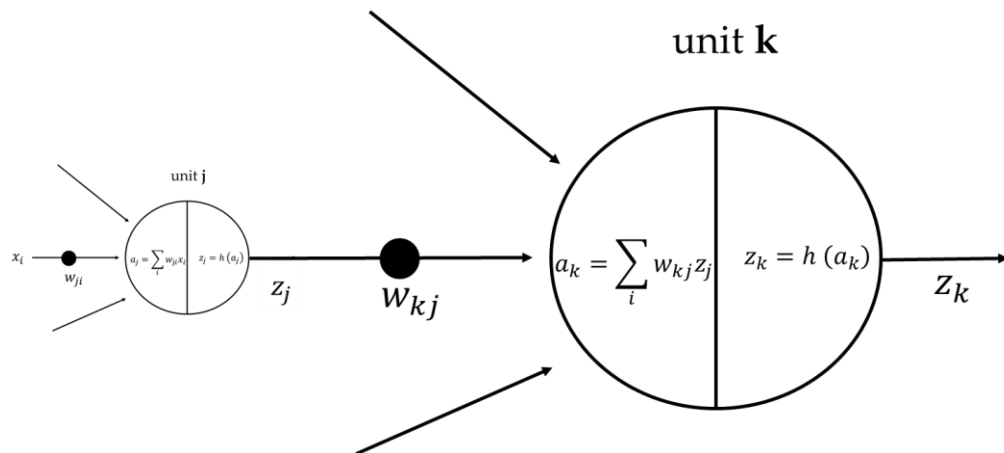
$$z_j = h(a_j)$$

Dica para lembrar a notação:

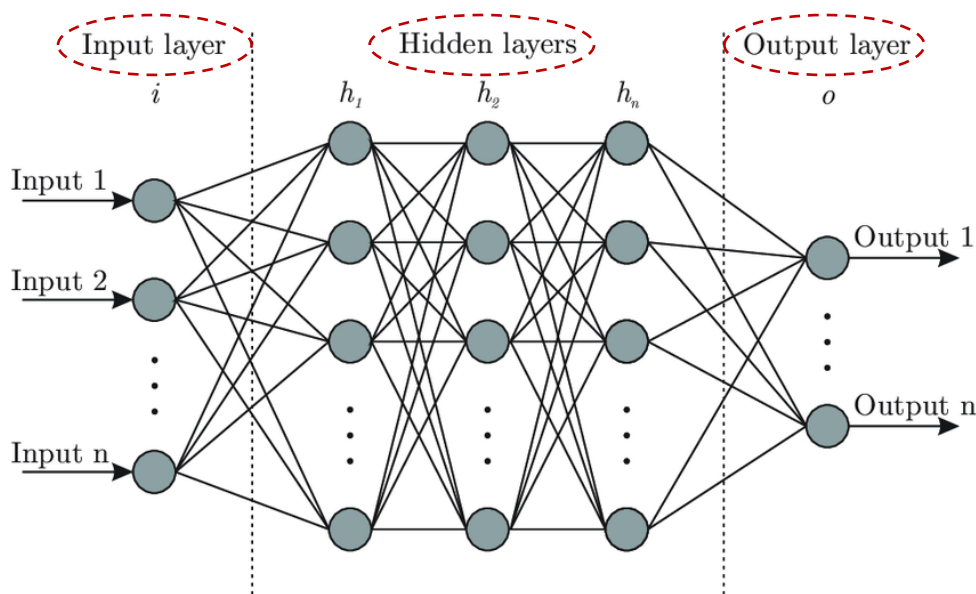
- os pesos são TO-FROM
- o primeiro índice é o da unidade que possui o peso





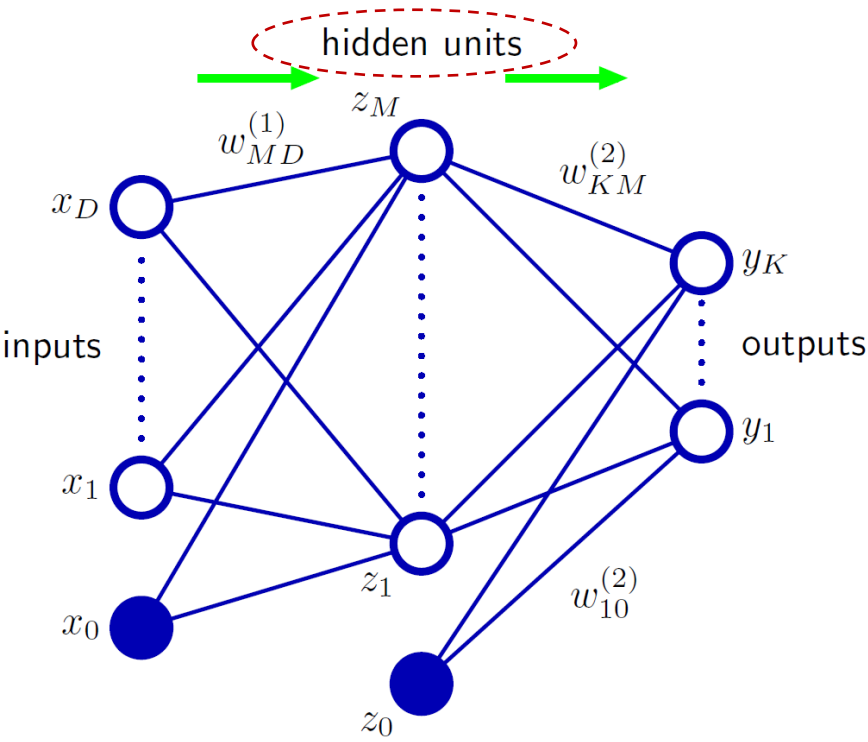


... e assim por
diante para
qualquer número
de camadas...



Vamos calcular a saída para uma determinada entrada

A propagação para a frente (rede de 2 camadas)



Observe a dimensão das matrizes de peso !!!

$\mathbf{W}^{(i)}$ matriz de pesos da camada (i)
 $h^{(i)}$ função de ativação da camada (i)

Entrada $x \in \mathbb{R}^D$; Saída $y \in \mathbb{R}^K$

N datapoints, um por linha, em $\mathbf{X}$ de tamanho $(N \times D)$

camada oculta : M unidades

camada de saída: K unidades

$\mathbf{W}^{(1)}$ é $(D \times M)$; $\mathbf{W}^{(2)}$ é $(M \times K)$

Tente escrever a fórmula da saída... pode omitir os biases

Saída da rede, um datapoint por linha

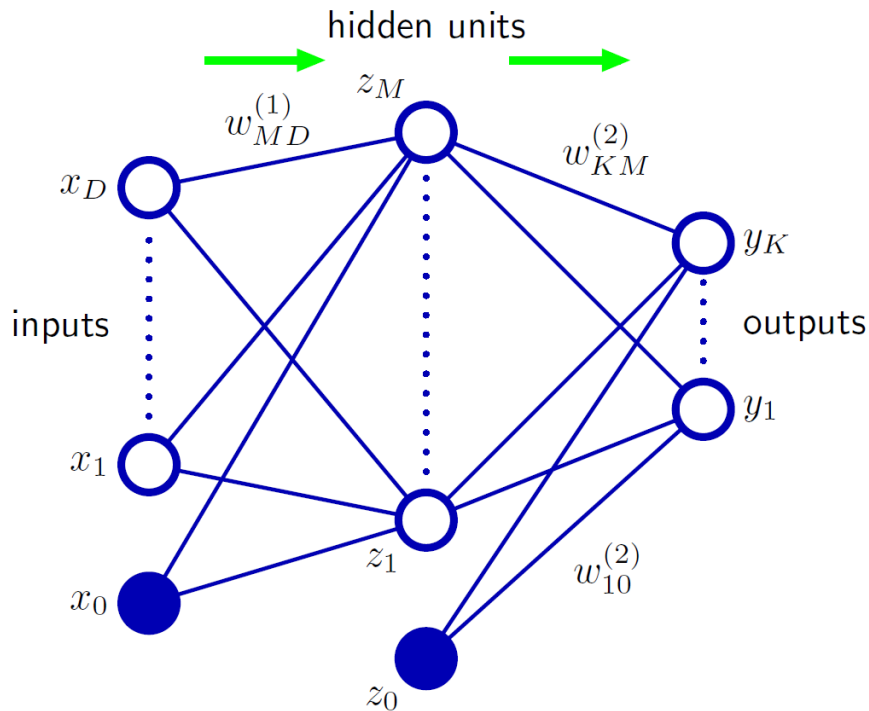
Saída da camada 1, um datapoint por linha

$$\mathbf{Y} = h^{(2)} \left(h^{(1)} \left(\mathbf{X} \cdot \mathbf{W}^{(1)} \right) \cdot \mathbf{W}^{(2)} \right)$$

(aqui omiti os biases – lembre-se de que eles precisam ser incluídos no conjunto de pesos)

Pontos importantes

1. Se todas as **funções de ativação das unidades ocultas forem lineares**, sempre podemos encontrar uma rede equivalente sem unidades ocultas.



$$\mathbf{Y} = \overset{\text{A}}{\downarrow} h^{(2)} \left(\mathbf{W}^{(2)} \cdot \overset{\text{B}}{\downarrow} h^{(1)} \left(\mathbf{W}^{(1)} \cdot \mathbf{X}^T \right) \right)$$

$$\mathbf{Y} = \mathbf{A} \cdot \mathbf{W}^{(2)} \cdot \mathbf{B} \cdot \mathbf{W}^{(1)} \cdot \mathbf{X}^T$$

onde A e B são matrizes

2. Múltiplas escolhas distintas para o vetor de peso w podem fornecer a mesma função.

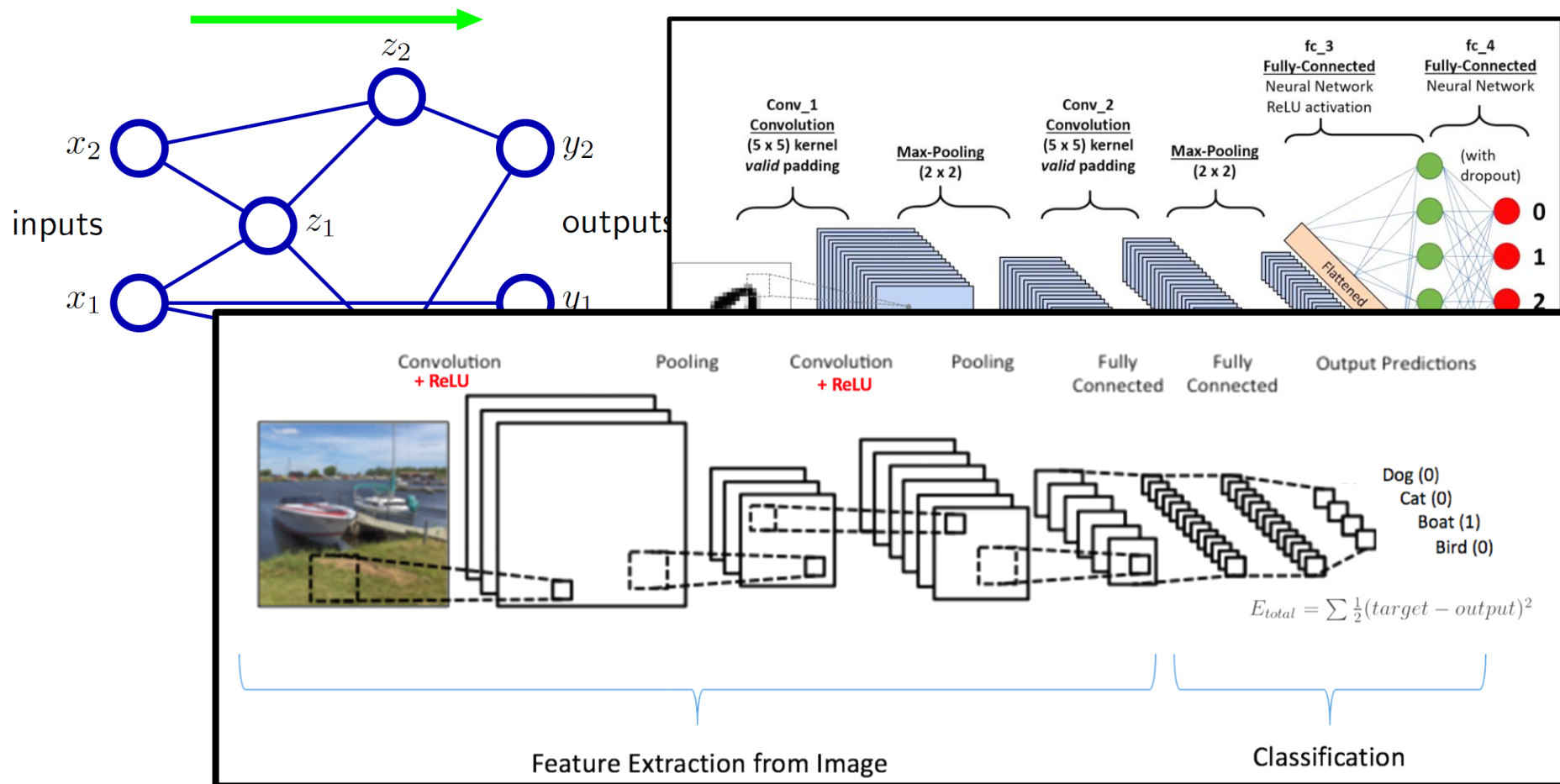
Para uma rede com uma camada oculta, M unidades:

- uma alteração no sinal de todos os pesos e os biases que alimenta uma unidade oculta específica pode ser compensada por uma alteração no sinal das saídas (2^M maneiras de fazer isso)
- podemos trocar os valores de todos os pesos que levam para dentro e para fora de uma unidade oculta específica com os valores correspondentes dos pesos associados a uma unidade oculta diferente ($M!$ maneiras de fazer isso)

A rede terá, portanto, um fator de simetria peso-espaco geral de $M! * 2^M$.

Para redes com mais de duas camadas de pesos, o nível total de simetria será dado pelo produto desses fatores, um para cada camada de unidades ocultas.

3. Como há uma correspondência direta entre um diagrama de rede e sua função matemática, **podemos desenvolver mapeamentos de rede mais gerais considerando diagramas de rede mais complexas.**

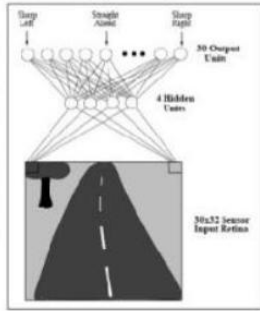




Uber ATG 2015

Autonomous driving

- ALVINN – Drives 70mph on highways

ALVINN CMU, 1989

4. *As redes neurais são aproximadores universais.* Uma rede de duas camadas com saídas lineares pode aproximar uniformemente qualquer função contínua em um domínio de entrada compacto com precisão arbitrária, desde que a rede tenha um número suficientemente grande de unidades ocultas (Hornik 1991).

Agora precisamos ter uma maneira de encontrar os pesos... **treinamento da rede.**

O que precisamos para treinar?

Uma função de erro!

Para problemas de regressão e classificação, precisamos usar diferentes **funções de ativação na unidades de saída.**

Roteiro

1. A ideia, as redes

-  2. Funções de erro e funções de ativação
3. Treinamento de rede neural
4. O algoritmo de retropropagação (backpropagation)
5. Regularização

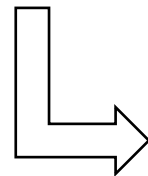
Uma interpretação probabilística para as saídas da rede nos dá uma motivação para :

1. escolha da **função de ativação da unidade de saída**
2. a escolha da **função de erro** .

... então começamos com isso ... 😊

(1) Regressão

Saída unidade k ativação função f : **identidade**


$$y = f\left(\sum_{j=1}^M w_{kj} z_j\right)$$

Pergunta: o que f deve ser?

Função de erro para minimizar:

Função de erro
SSE (soma de quadrados)

para uma única saída

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y_n - t_n)^2$$

para uma saída vetorial

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \sum_{k=1}^K (y_{nk} - t_{nk})^2$$

Pergunta: qual deve ser a função de erro?

N datapoints, k saída dimensional
 y_{nk} indica o k -ésimo componente de saída para o n -ésimo datapoint

... e é por causa disso...

$$p(t|\mathbf{x}, \mathbf{w}) = \mathcal{N}(t|y(\mathbf{x}, \mathbf{w}), \beta^{-1})$$

*t tem uma distribuição
Gaussiana, com média dada
pela saída da rede neural*

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = \prod_{n=1}^N p(t_n|\mathbf{x}_n, \mathbf{w}, \beta)$$

*função de
verossimilhança*

$$\frac{\beta}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2 - \frac{N}{2} \ln \beta + \frac{N}{2} \ln(2\pi)$$

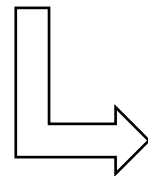
*Probabilidade de
log negativo*

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2$$

*Função de erro de
soma de
quadrados*

(2a) Classificação (caso binário)

Saída unidade k ativação função f : **sigmoide**


$$y = f\left(\sum_{j=1}^M w_{kj} z_j\right)$$

Pergunta: o que f deve ser?

Função de erro para minimizar: **entropia cruzada**

Pergunta: qual deve ser a função de erro?

há uma
única
saída

$$E(\mathbf{w}) = - \sum_{n=1}^N t_n \ln y_n + (1 - t_n) \ln(1 - y_n)$$

... e é por causa disso...

$$p(t|\mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w})^t \{1 - y(\mathbf{x}, \mathbf{w})\}^{1-t}$$

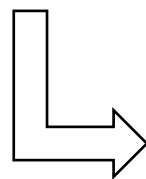
distribuição condicional de alvos

$$E(\mathbf{w}) = - \sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$

*Função de erro
de entropia
cruzada*

(2b) Classificação, k classificações separadas (classes não mutuamente exclusivas)

Saída unidade k ativação função f : **sigmoide**


$$y = f\left(\sum_{j=1}^M w_{kj} z_j\right)$$

Pergunta: o que f deve ser?

Função de erro para minimizar: **entropia cruzada**

Pergunta: qual deve ser a função de erro?

*existem
k saídas*

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk} + (1 - t_{nk}) \ln(1 - y_{nk})$$

N datapoints, k saída dimensional

y_{nk} indica o k -ésimo componente de saída para o n -ésimo datapoint

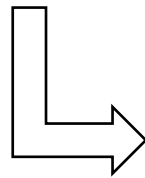
... e é por causa disso...

$$p(\mathbf{t}|\mathbf{x}, \mathbf{w}) = \prod_{k=1}^K y_k(\mathbf{x}, \mathbf{w})^{t_k} [1 - y_k(\mathbf{x}, \mathbf{w})]^{1-t_k}$$

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K \{t_{nk} \ln y_{nk} + (1 - t_{nk}) \ln(1 - y_{nk})\}$$

(2c) Classificação, k classes, mutuamente exclusivas

Saída unidade k ativação função f : **softmax**


$$y = f\left(\sum_{j=1}^M w_{kj} z_j\right)$$

Pergunta: o que f deve ser?

$$y_k = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$$

aqui j varia sobre as unidades de saída

Função de erro para minimizar: **entropia cruzada multiclasse**

existem k saídas

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}$$

Pergunta: qual deve ser a função de erro?

N datapoints, k saída dimensional

y_{nk} indica o k -ésimo componente de saída para o n -ésimo datapoint

... e é por causa disso...

$$p(\mathbf{T}|\mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{n=1}^N \prod_{k=1}^K p(\mathcal{C}_k|\phi_n)^{t_{nk}} = \prod_{n=1}^N \prod_{k=1}^K y_{nk}^{t_{nk}}$$

$$E(\mathbf{w}_1, \dots, \mathbf{w}_K) = -\ln p(\mathbf{T}|\mathbf{w}_1, \dots, \mathbf{w}_K) = -\sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}$$

a particular output unit takes the familiar form (5.18).

In summary, there is a natural choice of both output unit activation function and matching error function, according to the type of problem being solved. For regression we use linear outputs and a sum-of-squares error, for (multiple independent) binary classifications we use logistic sigmoid outputs and a cross-entropy error function, and for multiclass classification we use softmax outputs with the corresponding multiclass cross-entropy error function. For classification problems involving two classes, we can use a single logistic sigmoid output, or alternatively we can use a network with two outputs having a softmax output activation function.

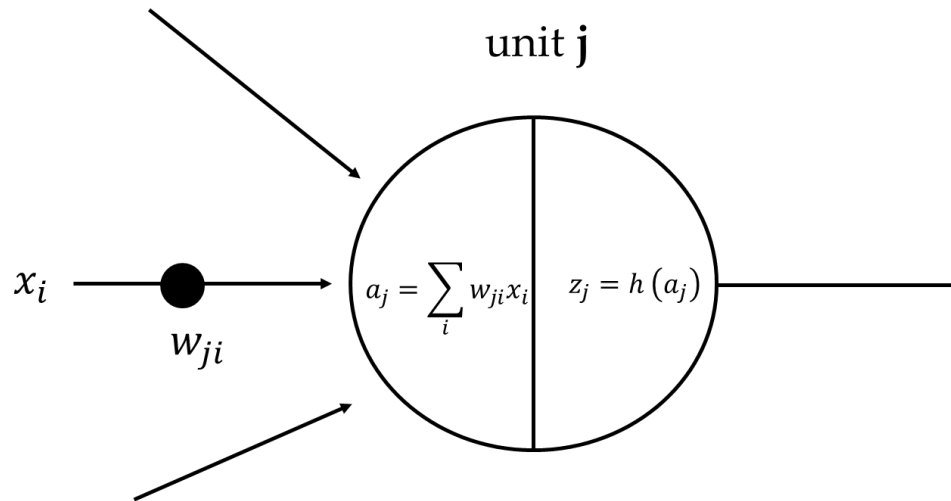
(C. Bishop, página 236)

Roteiro

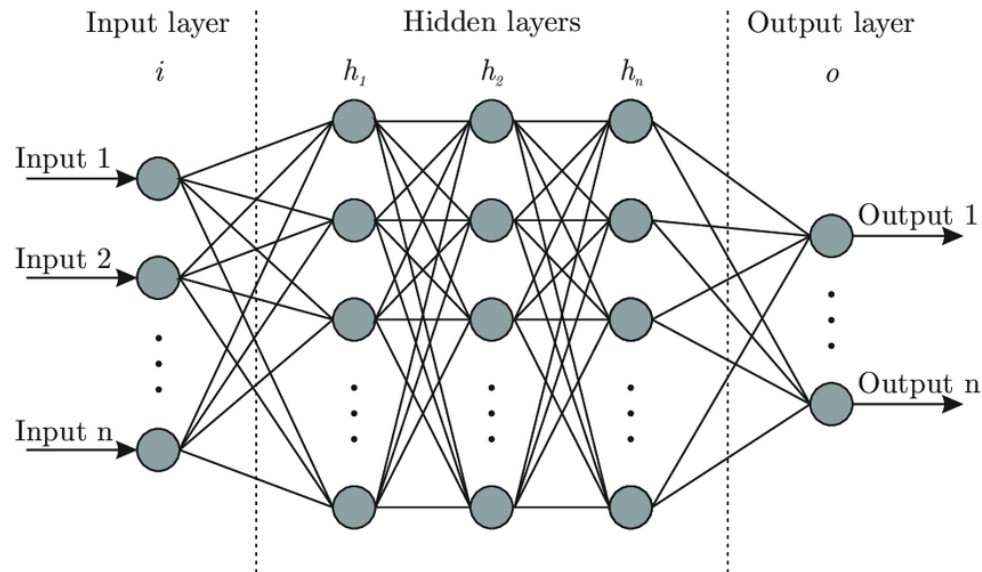
1. A ideia, as redes
2. Funções de erro e funções de ativação
-  3. Treinamento de rede neural
4. O algoritmo de retropropagação
5. Regularização

Treinamento de rede neural

Tarefa: aprender os pesos para minimizar uma função de erro.



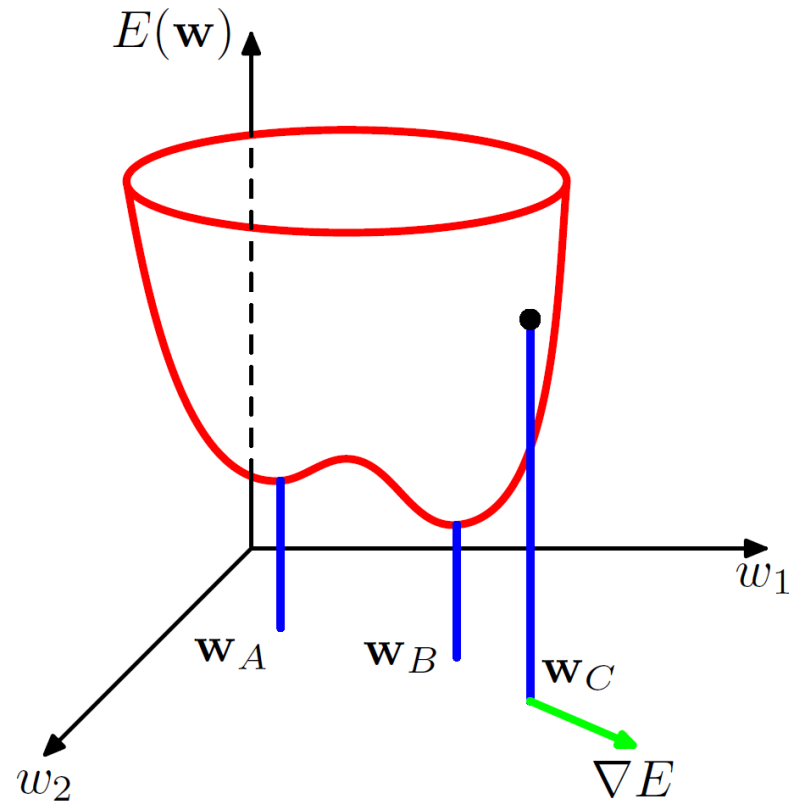
A grande questão era: como aprendo os pesos das unidades ocultas???



Minimizando a função de erro

*A função de erro é
uma superfície
assentada sobre o
espaço de peso.*

mínimo global
mínimos locais



Em geral, não temos solução de
forma fechada para:

$$\nabla E(\mathbf{w}) = 0$$

Nossa abordagem geral

Usamos procedimentos numéricos iterativos.

Escolha algum valor inicial $\mathbf{w}^{(0)}$ para o vetor de peso e, em seguida, mova-se pelo espaço de peso usando:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} + \Delta \mathbf{w}^{(\tau)}$$

- Diferentes algoritmos envolvem diferentes escolhas para a atualização do vetor peso $\Delta \mathbf{w}^{(\tau)}$.
- Muitos algoritmos fazem uso de **informações de gradiente**

DICA: plote o erro de aprendizagem durante o treinamento!

Otimização de descida de gradiente batch (ou descida mais íngreme – “gradient descent”)

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)})$$

η é chamada de *taxa de aprendizado*.

Método batch porque usa todo o conjunto de dados de uma só vez

Para otimização batch, existem métodos mais eficientes do que esta simples atualização (por exemplo, gradientes conjugados, métodos quase-Newton)

Otimização de descida de gradiente sequencial (ou estocástica – stochastic gradient descent)

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$$

onde $E_n(\mathbf{w})$ é o erro devido ao ponto de dados n . Então:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)})$$

faz uma atualização com base em um datapoint por vez.

Cenários intermediários: as atualizações são baseadas em lotes de pontos de dados.

Pode ser necessário executar um algoritmo baseado em gradiente várias vezes, com diferentes pontos de partida

Um procedimento iterativo para minimizar uma função de erro em uma rede neural

Cada passo tem 2 etapas:

1. Calcular as derivadas da função de erro em relação aos pesos
2. Usar as derivadas para calcular os ajustes dos pesos

Estágio 1 - backprop:

- pode ser aplicado a muitos tipos de rede.
- pode ser aplicado a diferentes funções de erro
- pode ser usado para calcular outras derivadas, como as matrizes jacobianas e hessianas.

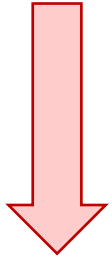
Backpropagation : um método eficiente para calcular derivadas.

Estágio 2: pode usar uma variedade de esquemas de otimização (também mais poderosos do que o gradiente descendente).

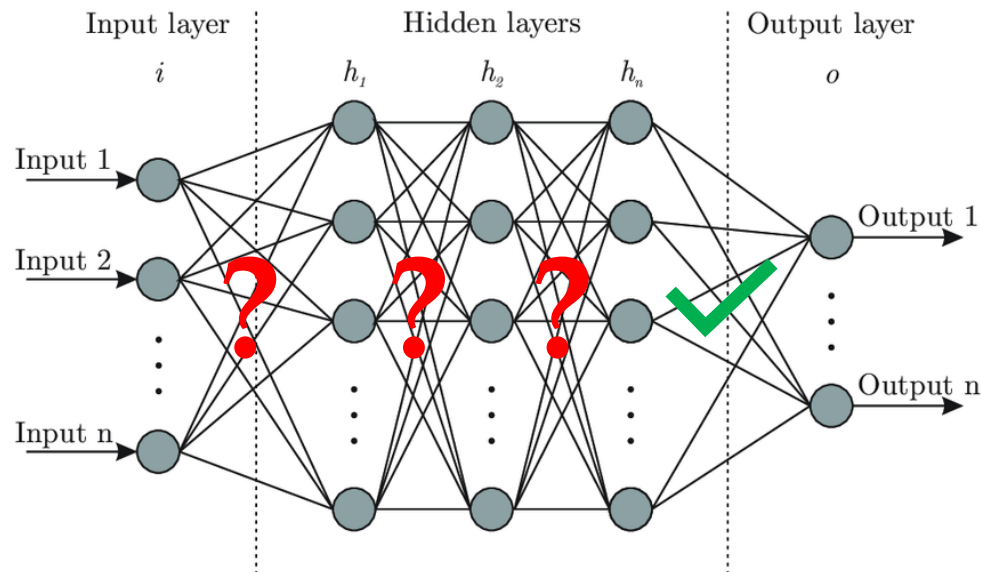
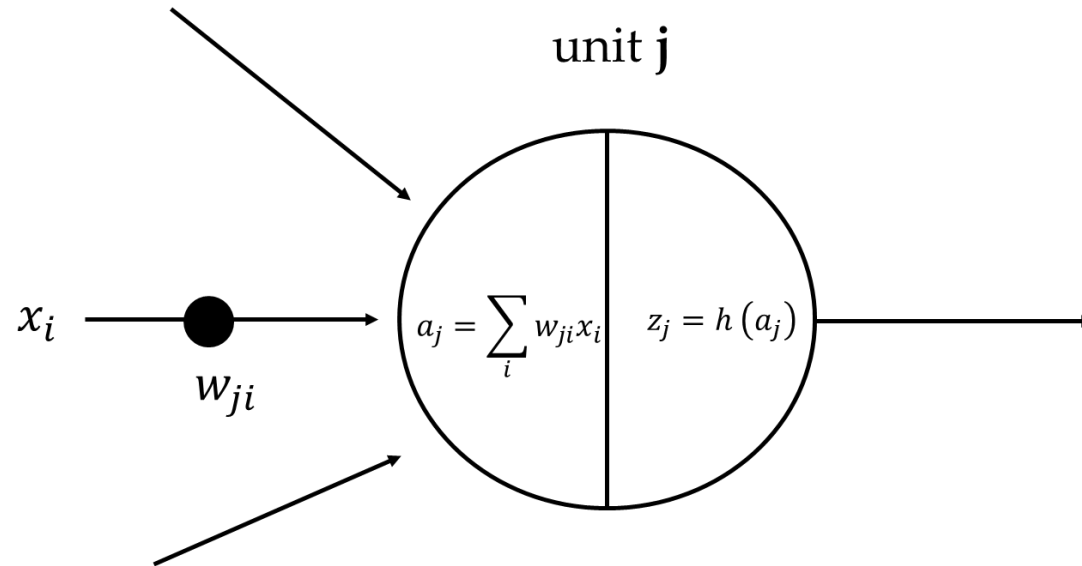
Roteiro

1. A ideia, as redes
2. Funções de erro e funções de ativação
3. Treinamento de rede neural
-  4. O algoritmo de retropropagação (backpropagation)
5. Regularização

Tarefa : aprender os pesos para minimizar uma função de erro.



$\frac{\partial E_n}{\partial w_{ij}}$ para qualquer peso w_{ij} na rede



O algoritmo de retropropagação

(caso de regressão)

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$$

$$y_k = \sum_i w_{ki} x_i$$

(ativação linear)

$$E_n = \frac{1}{2} \sum_k (y_{nk} - t_{nk})^2$$

para as unidades de saída

$$\frac{\partial E_n}{\partial w_{kj}} = (y_{nk} - t_{nk}) z_{nj}$$

Nota: uma computação “local” envolvendo o produto de um *signal de erro* e a variável de entrada.

para uma unidade genérica

$$a_j = \sum_i w_{ji} z_i$$

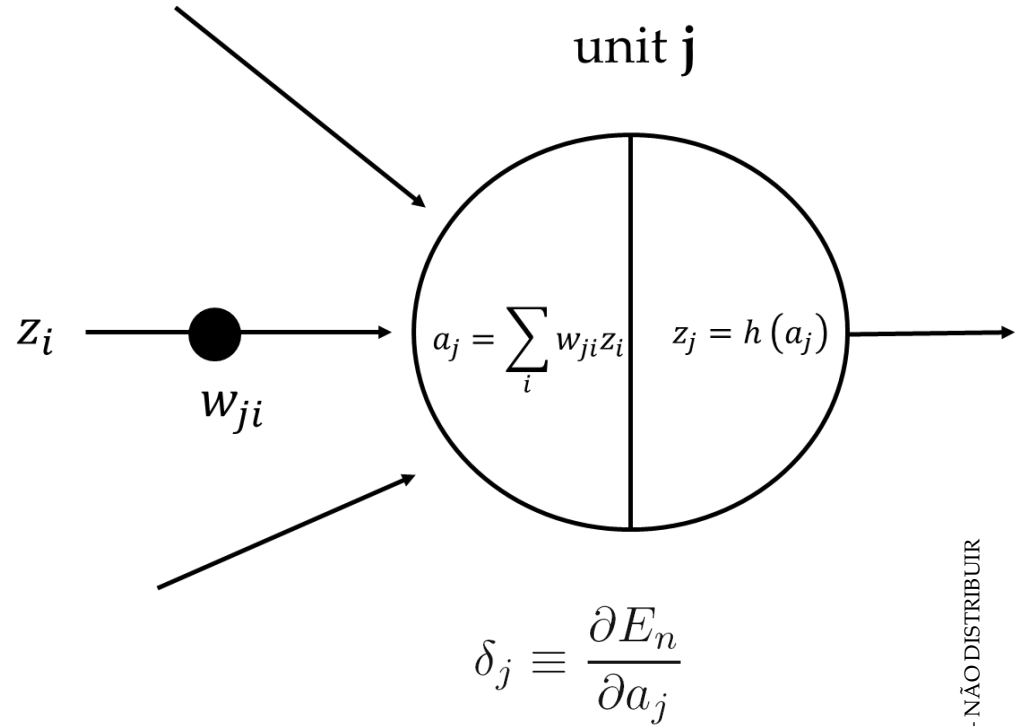
$$z_j = h(a_j)$$

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}}$$

$$\delta_j \equiv \frac{\partial E_n}{\partial a_j}$$

$$\frac{\partial a_j}{\partial w_{ji}} = z_i$$

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$



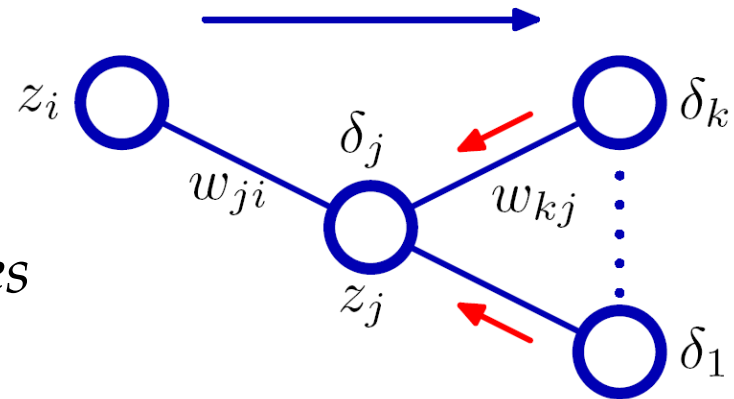
De fato, para as unidades de saída, nós já vimos que:

$$\delta_k = y_k - t_k$$

Para as unidades ocultas

$$\delta_j \equiv \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

onde a soma percorre todas as unidades k para as quais a unidade j envia conexões!



fórmula de
retropropagação:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

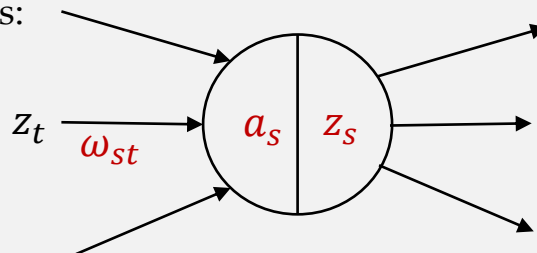
Vamos fazer mais uma vez...

Mostrarei os cálculos para um datapoint n (portanto, não o indicarei).

Para uma unidade genérica s:

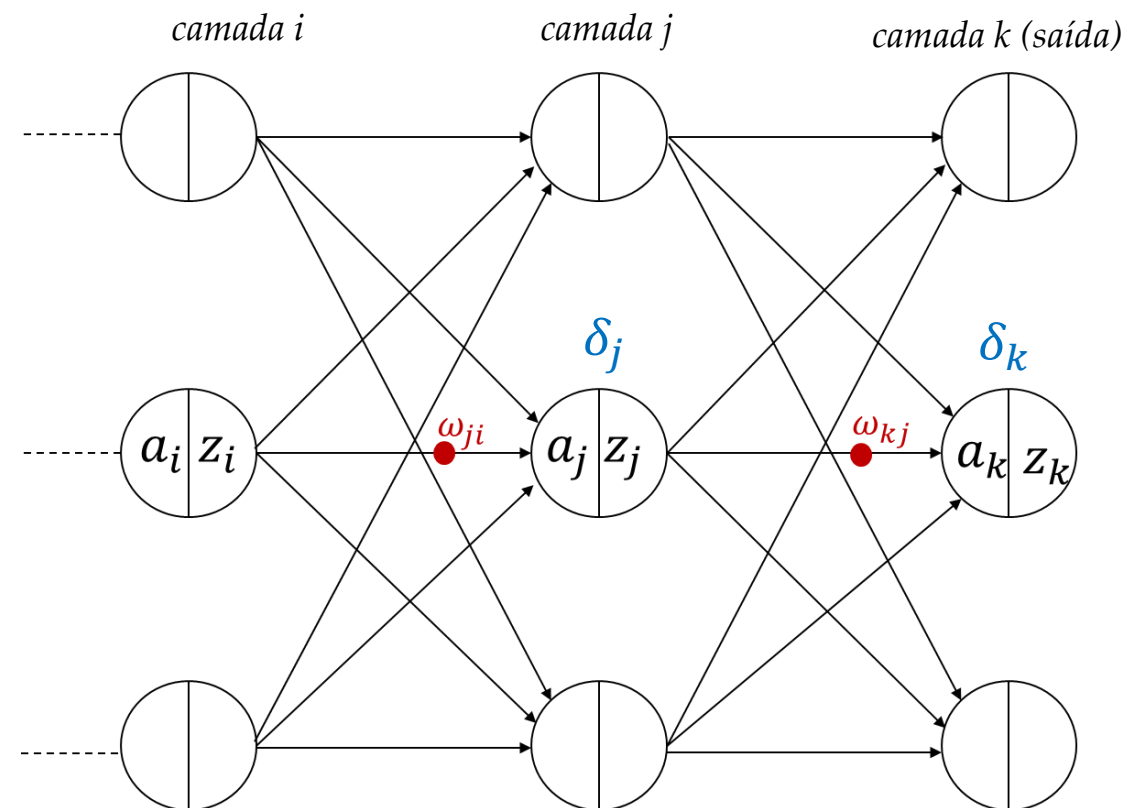
$$a_s = \sum_t w_{st} z_t$$

$$z_s = h(a_s)$$



onde h é a identidade na última camada e uma função diferenciável nas camadas ocultas

$$\frac{\partial E}{\partial w_{st}} = \underbrace{\frac{\partial E}{\partial a_s}}_{\delta_s} \cdot \underbrace{\frac{\partial a_s}{\partial w_{st}}}_{z_{\text{camada_anterior}}}$$



Para a camada de saída:

$$E = \frac{1}{2} \sum_k (a_k - t_k)^2$$

$$\delta_k = \frac{\partial E}{\partial a_k} = a_k - t_k$$

e portanto:

$$\frac{\partial E}{\partial w_{kj}} = (a_k - t_k) z_j$$

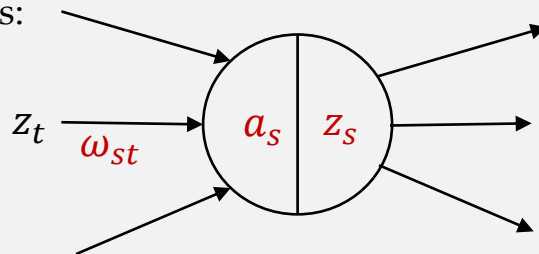
UFR

A.P

Para uma unidade genérica s:

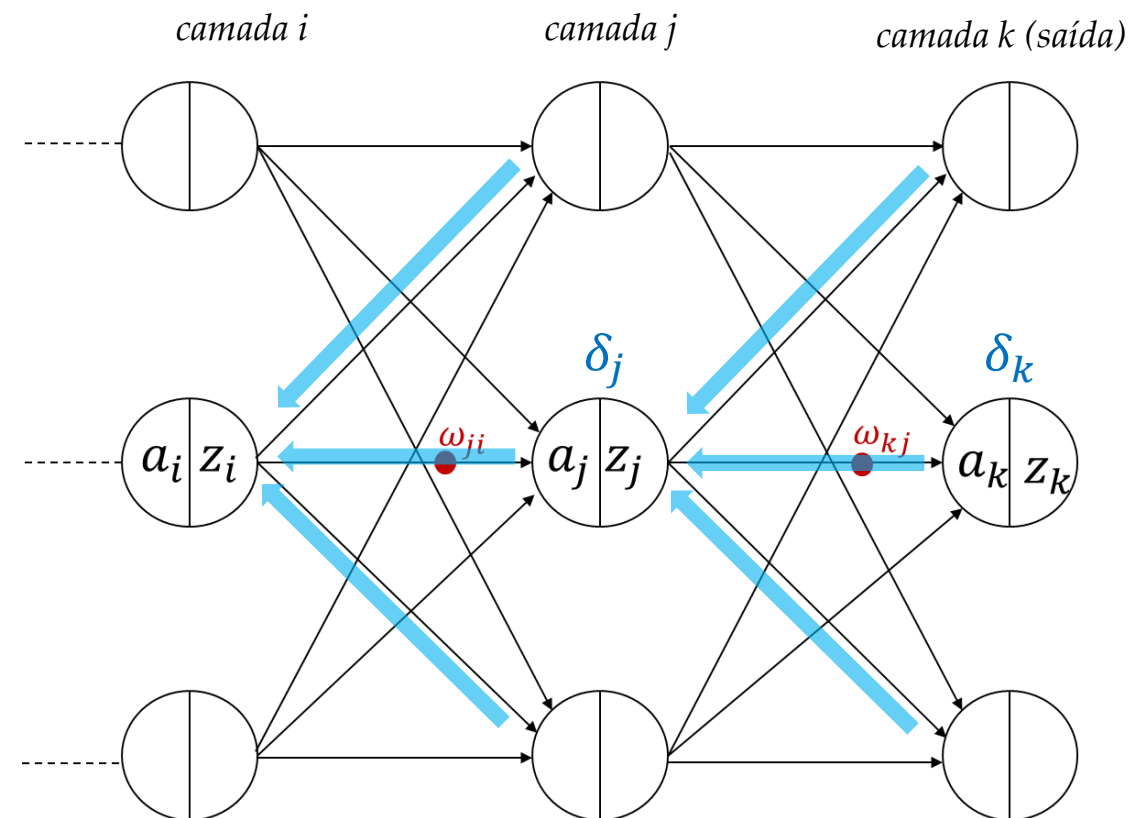
$$a_s = \sum_t w_{st} z_t$$

$$z_s = h(a_s)$$



$$\frac{\partial E}{\partial w_{st}} = \underbrace{\frac{\partial E}{\partial a_s}}_{\delta_s} \cdot \underbrace{\frac{\partial a_s}{\partial w_{st}}}_{z_{camada_anterior}}$$

onde h é a identidade na última camada e uma função diferenciável nas camadas ocultas



Para uma camada oculta:

UR

$$\delta_j \equiv \frac{\partial E}{\partial a_j} = \sum_k \frac{\partial E}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

$$a_k = \sum_j w_{kj} z_j = \sum_j w_{kj} h(a_j)$$

$$\frac{\partial a_k}{\partial a_j} = h'(a_j) w_{kj}$$

$$\delta_j = \sum_k \delta_k h'(a_j) w_{kj} = h'(a_j) \sum_k \delta_k w_{kj}$$

$$\frac{\partial E}{\partial w_{ji}} = \left(h'(a_j) \sum_k \delta_k w_{kj} \right) z_i \quad \text{A.P.}$$

Retropropagação de erro

1. Aplique um vetor de entrada $\mathbf{x}_n$ à rede e avance a propagação através da rede (*forward pass*)
2. Avalie o δ_k para todas as unidades de saída
3. Use a fórmula de retropropagação para retropropagar os δ e obter δ_j para cada unidade oculta (*backward pass*)
4. As derivadas necessárias são dadas por $\delta_j z_i$ onde z_i é a i -ésima entrada da unidade j .

Importante: veja o exemplo na página 245 do livro de Bishop (seção 5.3.2)

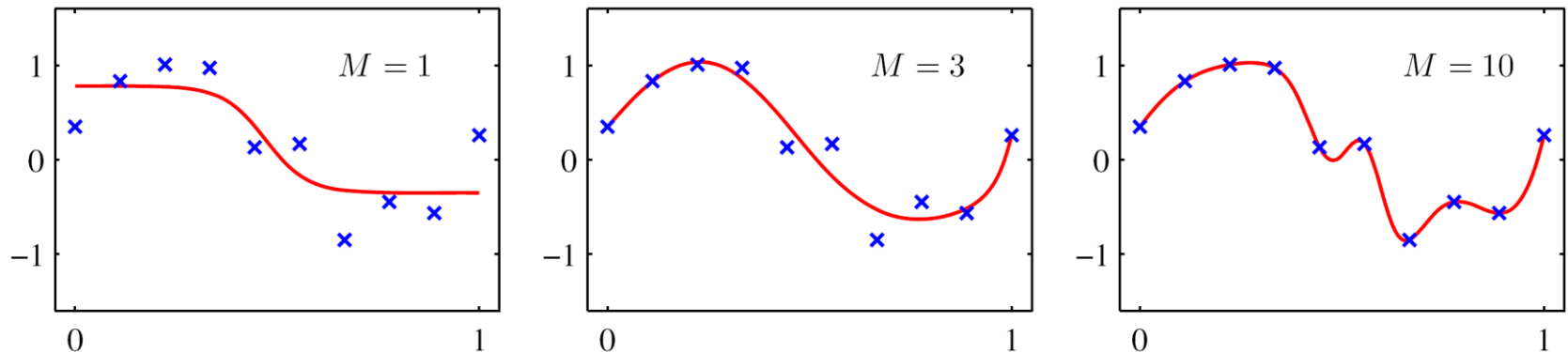
Roteiro

1. A ideia, as redes
2. Funções de erro e funções de ativação
3. Treinamento de rede neural
4. O algoritmo de retropropagação (backpropagation)
-  5. Regularização

Número de parâmetros livres

O número de unidades controla o número de parâmetros (pesos)

- **Número de unidades de entrada e saída:** determinado pela dimensão do conjunto de dados (de entrada e saída)
- **Número M de unidades ocultas:** parâmetro livre, escolhido pelo usuário



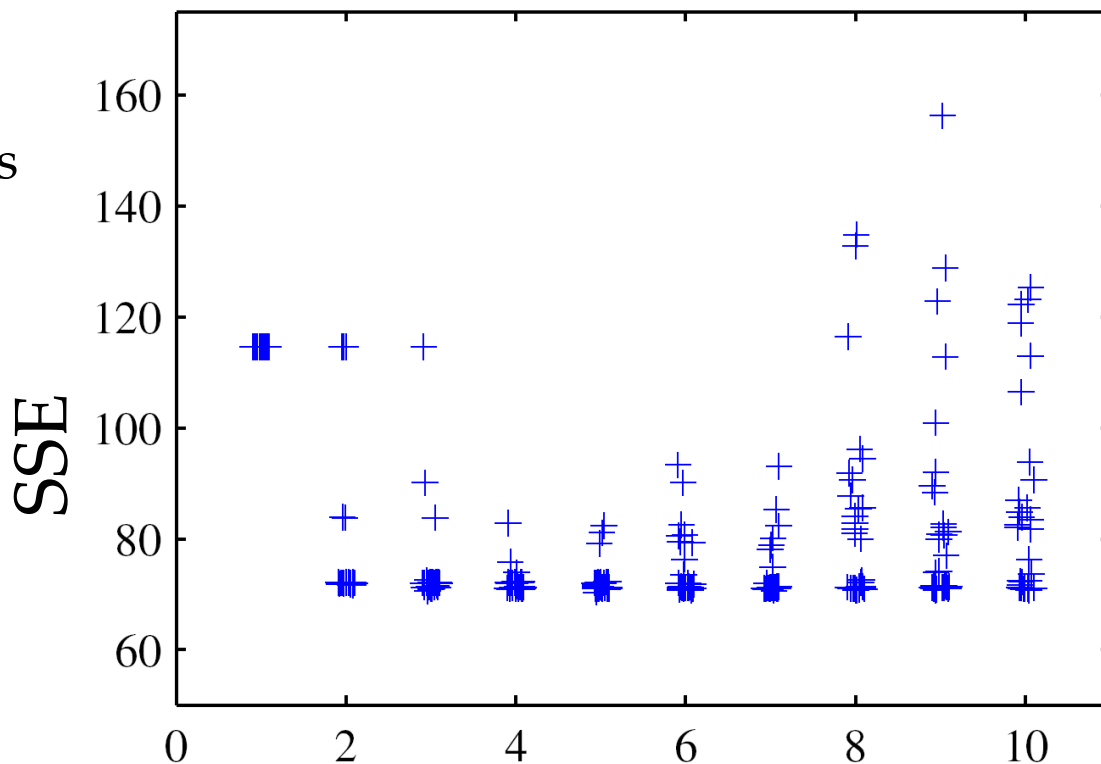
Mínimos Locais

O erro de generalização não é uma função simples de M devido aos mínimos locais na função de erro

Desempenho para o exemplo de aproximação polinomial
no **conjunto de validação**

Efeito de várias
inicializações aleatórias
para um intervalo de
valores de M .

*(uma maneira de
escolher M é plotar
esses gráficos) .*



Regularização – *weight decay*

Podemos controlar a complexidade adicionando um termo de regularização à função de erro.

O regularizador mais simples é o quadrático:

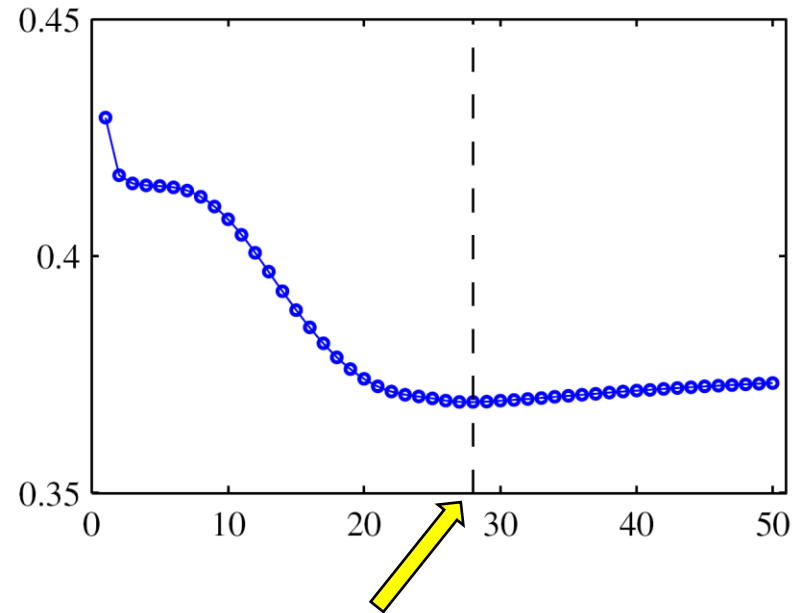
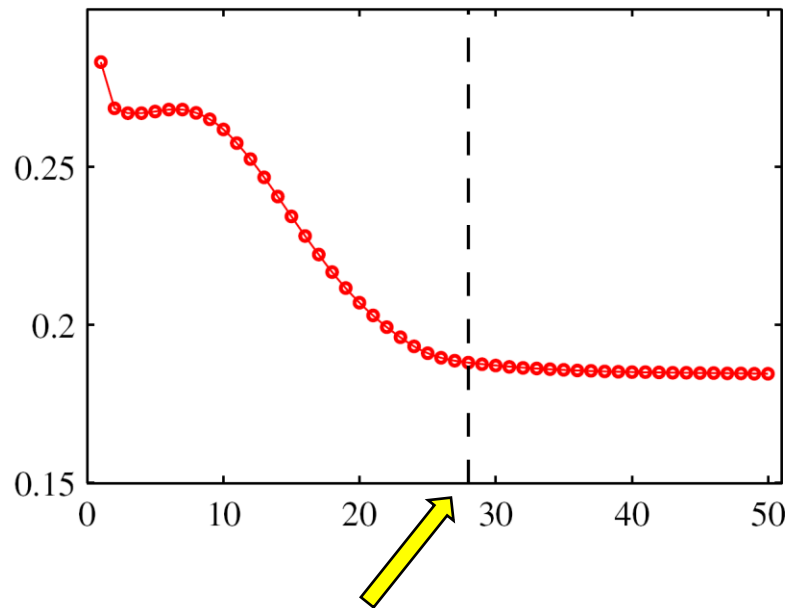
$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$$

Esse regularizador pode ser interpretado como o logaritmo negativo de um prior gaussiano de média zero sobre o vetor de peso w .

Regularização – *early stopping*

O erro medido em relação a um conjunto de validação geralmente mostra uma diminuição no início, seguida de um aumento à medida que a rede começa a se sobreajustar.

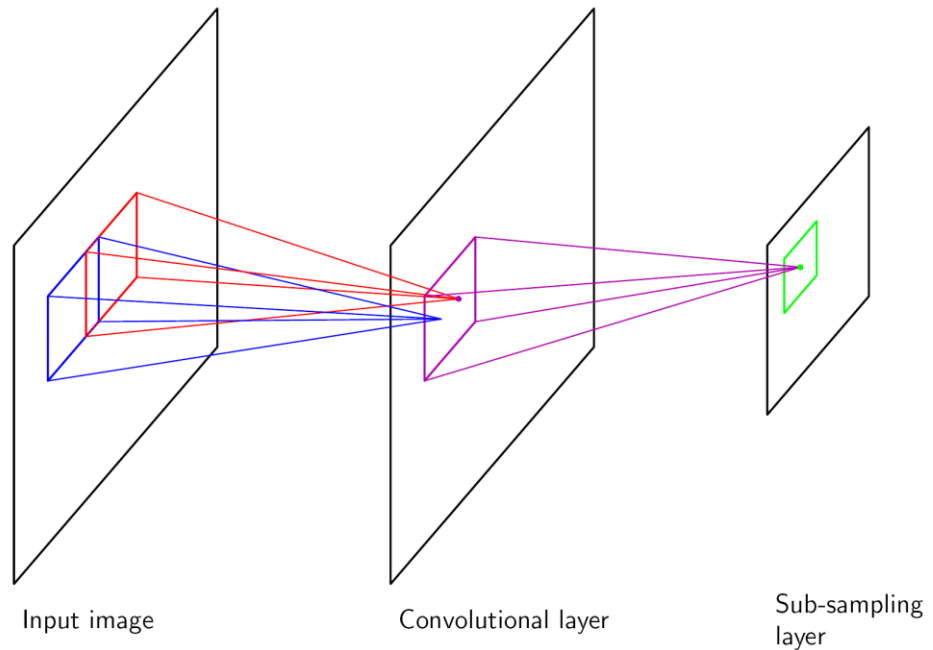
Por que?



O treinamento pode, portanto, ser interrompido no ponto de menor erro em relação ao conjunto de dados de validação

Redes convolucionais

Na camada convolucional, as unidades são organizadas em *feature maps*.



As unidades em um mapa de recursos recebem entradas de uma pequena sub-região

Todas as unidades em um mapa de recursos são restritas para compartilhar os mesmos valores de peso.